CI/CD for LLM Products:

The Continuous Improvement Flywheel

Start With Error Analysis & AI Evals

01. Develop & Analyze Errors

Start with initial LLM pipeline development (prompts, RAG setup, tool integrations). Perform thorough **Error Analysis** and construct a **Golden Dataset**.

02. Measure & Build Evals

Translate qualitative failure modes into **quantifiable metrics**. Implement and validate a suite of **automated evaluators** (code-based and LLM-as-Judge).

03. Continuous Integration Setup

Integrate the golden dataset and automated evaluators into the **CI pipeline** to act as **regression tests** for known issues.

04. Deploy (CD) with Observability

Ship the LLM pipeline instrumented for comprehensive logging. Deploy automated evaluators to run on **sampled production** traffic (often asynchronously).

How to Perform Error Analysis?

The most successful teams look at data, identify failure modes, and let **app-specific metrics** emerge bottom-up.

You analyze **LLM traces - full records** of the pipeline: user query, input, reasoning, tool calls, and the output.

1 Add New Traces (Real or Synthetic)

2 Read And Open Code Traces

3 Axial Coding: Refine Failure Modes

4 Re-Code Traces With Failure Modes

Theoretical saturation: Repeat until no new failure modes & no changes in re-coding appear

You need **~100** high-quality, diverse traces. Those can be real or synthetic data, or both coded with pass/fail and failure modes.

How to Build Automated Evals?

Once you perform initial error analysis, you can implement **automated evaluators** for single failure modes - each **evaluating a single metric**.

A. Start With Analyzing Failure Type

Specification Failure

- Your instructions were unclear or incomplete
- **Fix the prompt** first. Don't build an evaluator yet

Generalization Failure

- LLM fails to apply clear instructions correctly
- These are candidates for **automated evaluators**

B. Consider Two Types of App-Specific Evaluators

Code-Based Evals

- Logic you write (e.g., .py)
- Objective, rule-based checks such as XM or Regex
- **Fast, cheap,** deterministic

LLM-as-Judge Evals

- Complex or subjective tests
- Uses another LLM as judge
- Single, narrow failure mode
- Start with **binary checks**

What is a Golden Dataset?

It's the foundation of LLM CI. A hand-curated set of input examples with corresponding reference outputs. Typically 100+ pairs.

On each proposed change we run all golden inputs through the pipeline and evaluate the outputs with a automated evaluators.

A Golden Dataset should:

- Cover the core features and capabilities
- Represent critical failures modes observed in the past
- Include edge cases
- Measure different elements of the pipeline (e.g., tools, RAG)

The Continuous Improvement Flywheel Continues

05. Monitor Online Performance

Actively track the corrected success rates and confidence intervals for **key failure modes** (production). Dashboard these metrics and **set up alerts.** Also, track product metrics.

06. Drift, New Failures, or Issues

Analyze **online evals data**. Review relevant **product metrics** (e.g., low CSAT might indicate an LLM issue). Keep **humans in the loop** to hunt for "unknown unknowns"

?

07. Re-Analyze (Error Analysis)

When a new **significant failure, issue or drift** is confirmed (via automated evals, product metrics, or human review), conduct targeted **Error Analysis**.

08. Update Evaluation Artifacts

Include **major failure modes**:

- Golden Dataset
- Evaluators & Rubrics
- Regularly verify model-human alignment
- CI jobs (regression tests)

09. Improve LLM Pipeline

Refine prompts, decompose complex tasks, adjust RAG strategies, improve tool schemas and descriptions, refine guiderails, consider fine-tuning (judiciously)

10. Re-Deploy

Ship the improved pipeline and **updated evaluation artifacts**. Resume monitoring, expecting the targeted **success rates to increase**.

When to Repeat Error Analysis?

Some ideas:

- After substantial product changes (e.g., new agent capabilities like DB querying, different feature goals).
- If user demographics or query distributions shift significantly.
- If online evaluators show persistent new failure patterns or product metrics indicate an AI problem.

The goal here is to understand the root causes and whether the current eval artifacts remain relevant.

What Are Guiderails?

Evaluators that run synchronously during pipeline execution, before producing a final output to the user.

Typically fast and reliable code-based checks such as regex filters of checking against a blacklist. Possible responses: **Reject, Retry, Fallback**.